Advanced Machine Learning, ETH Zurich

Attention

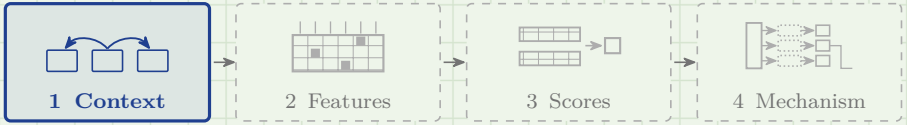
How a word reads the words around it

Carlos Cotrini

Autumn semester 2026

Entry 1

Words in context



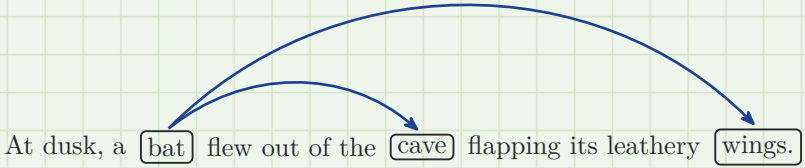
The question. In the language model, every row of E holds one word, yet the next word depends on the whole prefix. So row i must read the rows above it. How?

Now. Attention: every row reads the other rows and adds to itself what it finds. In Wuggish, the toy language of this lecture, every word finds what it needs in the rows above it.

The same word can mean two different things. The words around it decide which one.

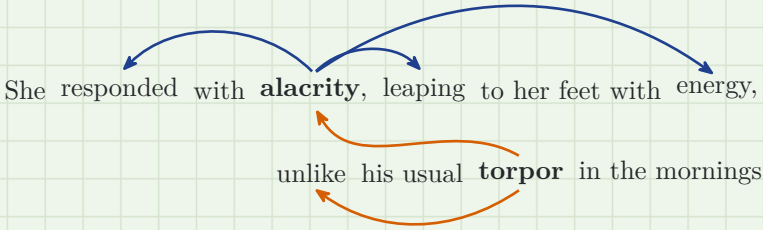


Here **bat** reads baseball and hit: a club.



Here **bat** reads cave and wings: an animal.

Few people know the words alacrity and torpor. Read the sentence: its other words tell you what they mean.



alacrity reads responded, leaping and energy: something quick and lively.

torpor reads unlike and alacrity: the opposite of alacrity.



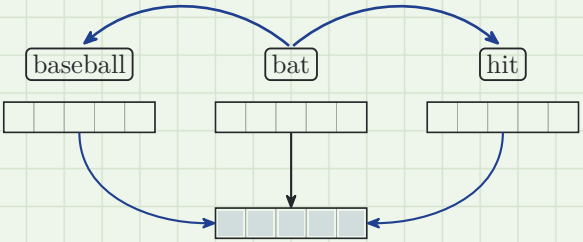
alacrity: brisk and cheerful readiness.



torpor: a state of physical or mental inactivity.

A network that reads a sentence does it in two steps, and every word does both.

1. Every word gets a representation: a row of numbers.



bat, now the club

2. Every word reads the other words and adds to its row what it finds.

This is *attention*. The rest of the lecture computes it by hand, on a language small enough to see every number.

Entry 2

Wuggish



So far. A word takes its meaning from the words around it. Attention lets every word read the other words and add what it finds to its own row.

Now. A toy language of eleven words, Wuggish, and a way to write each word as numbers: a row of seven features, each a 1 or a 0.

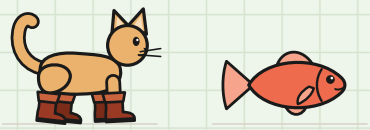
Wuggish is a made-up sister language of English. Its sentences describe a cat, a duck and a fish, each plain or with a look: running, in boots, swimming or diving.

		runnin	*bootet*	*swimmin*	*divin*
kat			 ***pussinbootz***		
dukk		 ***runnerdukk***		 ***pondukk***	
feesh					 ***divefeesh***

Eleven words: three things, four looks, and four things with a look. *pussinbootz* is the cat in boots that *bootet kat* draws.

Learn the eleven words from pictures, then read 12 sentences whose words come in a fresh order every time. At the end, the game shows what you have been doing all along.

bootet *feesh* *kat*



Play it now, on a laptop or a phone: scan the code, or open the address below.

eth-aml-hs26.github.io/neural-networks/viz/wuggish-game

To compute with words, we ask every word the same seven questions. Each question is a *feature*, and its answer is 1 for yes and 0 for no.

What a thing has

LEGS has legs? 

FINS has fins? 

WINGS has wings? 

The nouns say yes here: a cat has legs, a duck legs and wings, a fish fins.

Where they meet

earth

water

A cat runs and wears boots on earth, a fish swims and dives in water, and a duck does all four.

Earth and water get no column of their own.

What a look brings

RUNNING is running? 

BOOTS wears boots? 

SWIMMING is swimming? 

DIVING is diving? 

An adjective brings its look, and so does a word that names a thing with a look.

Ask every Wuggish word the seven questions and write its answers in a row, its *embedding*. The eleven rows form the *embedding table*.

	has			brings			
	LEGS	FINS	WINGS	RUNNING	BOOTS	SWIMMING	DIVING
kat	**1**	0	0	0	0	0	0
dukk	**1**	0	**1**	0	0	0	0
feesh	0	**1**	0	0	0	0	0
runnerdukk	**1**	0	**1**	**1**	0	0	0
pussinbootz	**1**	0	0	0	**1**	0	0
pondukk	**1**	0	**1**	0	0	**1**	0
divefeesh	0	**1**	0	0	0	0	**1**
runnin	0	0	0	**1**	0	0	0
bootet	0	0	0	0	**1**	0	0
swimmin	0	0	0	0	0	**1**	0
divin	0	0	0	0	0	0	**1**

11 words by $d = 7$ features

kat, *dukk* and *feesh* say what they have: legs; legs and wings; fins.

runnerdukk, *pussinbootz*, *pondukk* and *divefeesh* each add one look: Puss in Boots is a cat in boots.

An adjective says what its look brings, with a single 1.

We wrote these rows by hand, to read them. A trained network learns its own rows, and its features have no names.

A word of a sentence, a *token*, is a row of the table. A feature is a column: a question that several words answer with yes.

	has			brings			
	LEGS	FINS	WINGS	RUNNING	BOOTS	SWIMMING	DIVING
kat	**1**	0	0	0	0	0	0
dukk	**1**	0	**1**	0	0	0	0
feesh	0	**1**	0	0	0	0	0
runnerdukk	**1**	0	**1**	**1**	0	0	0
pussinbootz	**1**	0	0	0	**1**	0	0
pondukk	**1**	0	**1**	0	0	**1**	0
divefeesh	0	**1**	0	0	0	0	**1**
runnin	0	0	0	**1**	0	0	0
bootet	0	0	0	0	**1**	0	0
swimmin	0	0	0	0	0	**1**	0
divin	0	0	0	0	0	0	**1**
words	**5**	**2**	**3**	**2**	**2**	**2**	**2**

A token: the row of *pussinbootz*, seven numbers that say a cat in boots.

A feature: the column LEGS. Five words say yes, *kat*, *dukk*, *runnerdukk*, *pussinbootz* and *pondukk*: legs are what they share.

Every column holds the 1s of two words or more: the count under it.

And there are more words than features: 11 rows, 7 columns. So a feature can never be a second name for a word.

Two rows of the table add up to a third. The row of *pussinbootz* holds the row of *kat* and one more 1, at BOOTS: the row of *bootet*.

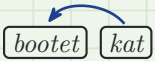
	LEGS	FINS	WINGS	RUNNING	BOOTS	SWIMMING	DIVING
row of *kat*	1	0	0	0	0	0	0
+ row of *bootet*	0	0	0	0	1	0	0
row of *pussinbootz*	1	0	0	0	1	0	0



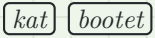
The other three alike: *runnerdukk* is *dukk* plus *runnin*, *pondukk* is *dukk* plus *swimmin*, and *divefeesh* is *feesh* plus *divin*.

Keep this sum in mind: in our sentence, attention will add the row of *bootet* to the row of *kat*.

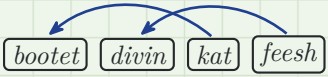
Wuggish has one rule of order: an adjective stands somewhere before the noun it describes. Other words may stand between them, and beyond that the order carries nothing.



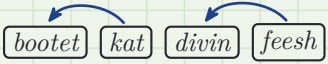
✓ a cat in boots



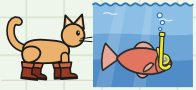
✗ the adjective stands after its noun



✓ other words between them



✓ the sentence we compute



So a noun finds its adjectives among the words before it: in E, in the rows above its own.

Write our sentence one word per row and replace every word by its row of the table. The result is E, with $n = 4$ rows and $d = 7$ columns.

	LEGS	FINS	WINGS	RUNNING	BOOTS	SWIMMING	DIVING
bootet	0	0	0	0	**1**	0	0
kat	**1**	0	0	0	0	0	0
divin	0	0	0	0	0	0	**1**
feesh	0	**1**	0	0	0	0	0

E: $n = 4$ rows, $d = 7$ columns

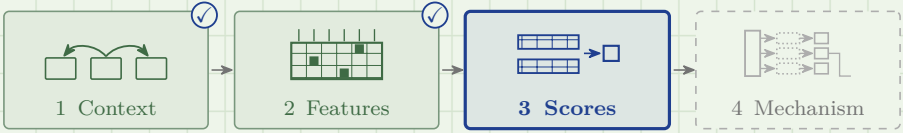
The row of *kat* holds LEGS alone: a cat. Nothing in it says booted, and *bootet* stands in the row above. Attention will make the row of *kat* read the row of *bootet*.

	LEGS	FINS	WINGS	RUNNING	BOOTS	SWIMMING	DIVING
kat	**1**	0	0	0	0	0	0
dukk	**1**	0	**1**	0	0	0	0
feesh	0	**1**	0	0	0	0	0
runnerdukk	**1**	0	**1**	**1**	0	0	0
pussinbootz	**1**	0	0	0	**1**	0	0
pondukk	**1**	0	**1**	0	0	**1**	0
divefeesh	0	**1**	0	0	0	0	**1**
runnin	0	0	0	**1**	0	0	0
bootet	0	0	0	0	**1**	0	0
swimmin	0	0	0	0	0	**1**	0
divin	0	0	0	0	0	0	**1**

the embedding table

Entry 3

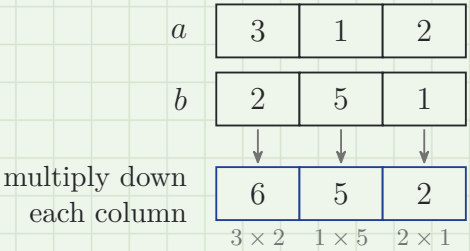
Queries, keys and scores



So far. Every word is a row of seven features, and our sentence is a block E. In E, the row of *kat* says nothing about boots.

Now. Every word must find the rows to add to its own. We recall the game, picture it as a shopping mall, and let queries and keys find the rows.

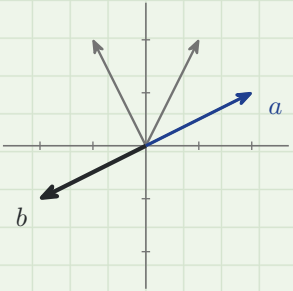
Two rows of the same length give one number, their *inner product*.



then add $6 + 5 + 2 = \boxed{13}$ the inner product of a and b

inner product of a and $b = a_1b_1 + a_2b_2 + a_3b_3$

Read a row of two numbers as an arrow. Keep a and turn b: the inner product falls as the angle between them opens.

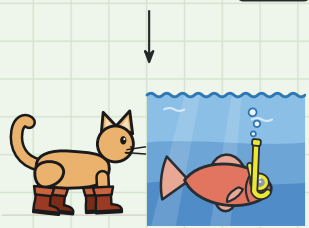


$a = (2, 1)$

$b = (2, 1)$	$2 \times 2 + 1 \times 1 = 5$	same direction
$b = (1, 2)$	$2 \times 1 + 1 \times 2 = 4$	close directions
$b = (-1, 2)$	$2 \times (-1) + 1 \times 2 = 0$	at right angles
$b = (-2, -1)$	$2 \times (-2) + 1 \times (-1) = -5$	opposite directions

Think back to the game. Our sentence *bootet kat divin feesh* draws a cat in boots and a fish that dives. How did you find that picture?

bootet *kat* *divin* *feesh*



next animal

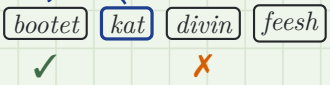
1 Take the next animal of the sentence.

2 Collect the adjectives that fit it.

3 Give it their looks: add their rows to its own row.

Our sentence has two animals, so you went round the loop twice: first for *kat*, then for *feesh*.

Round one of the loop. The first animal of our sentence is *kat*: a cat, which walks on earth.



	LEGS	FINS	WINGS	RUNNING	BOOTS	SWIMMING	DIVING
kat	1	0	0	0	0	0	0
+ *bootet*	0	0	0	0	1	0	0
= *pussinbootz*	1	0	0	0	1	0	0

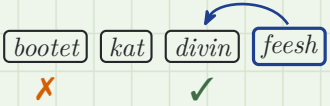


1 Take the next animal: *kat*.

2 Collect the adjectives that fit it. *bootet* fits: boots go on earth. *divin* stays out: diving happens in water.

3 Add their rows to its own. *kat* gains BOOTS: the row of *pussinbootz*, the cat in boots.

Round two of the loop. The next animal is *feesh*: a fish, which swims in water.



	LEGS	FINS	WINGS	RUNNING	BOOTS	SWIMMING	DIVING
feesh	0	**1**	0	0	0	0	0
+ *divin*	0	0	0	0	0	0	**1**
= *divefeesh*	0	**1**	0	0	0	0	**1**



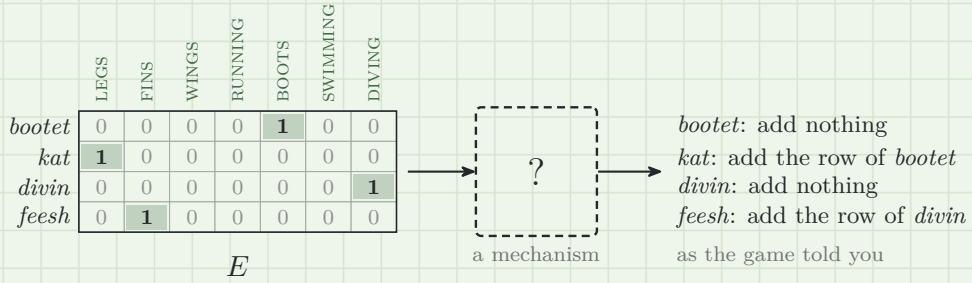
1 Take the next animal: *feesh*.

2 Collect the adjectives that fit it. *divin* fits: diving happens in water. *bootet* stays out: boots go on earth.

3 Add their rows to its own. *feesh* gains DIVING: the row of *divefeesh*, the fish that dives.

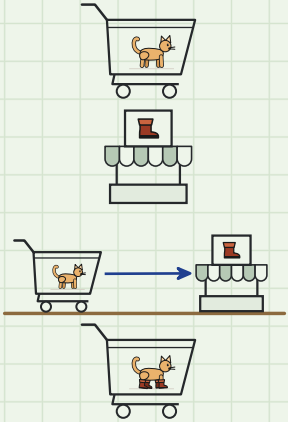
No animal is left: the loop ends, and the sentence is drawn.

A network sees no pictures and knows no rules of a game. It has only the numbers of E, and from them alone it must find, for every word, the rows to add to its own.



In the game the pictures told you which looks fit. A network needs a mechanism that reads the same answers off the numbers of E, for any sentence.

To find that mechanism, picture a sentence as a shopping mall. Its nouns go shopping, and its adjectives sell.



Every **noun** is a shopper with a cart, and its own animal is in the cart already: *kat* comes with a plain cat.

Every **adjective** is a stand, with a sign that shows what it sells: *bootet* sells boots.

A shopper buys at the stands that sell what it looks for. A cat shops on earth, and a fish shops in water.

At the exit, the cart holds the new row: *kat* leaves as a cat in boots.

You go shopping for a party. What you look for is the *query*; the signs over the shops are the *keys*; what you carry home are the *values*.

Query



Keys

GreenChoice Supermarket



StoryStack Books



Threadline Clothing Co



Candy Cave



Spirit Cellar Liquors



Values







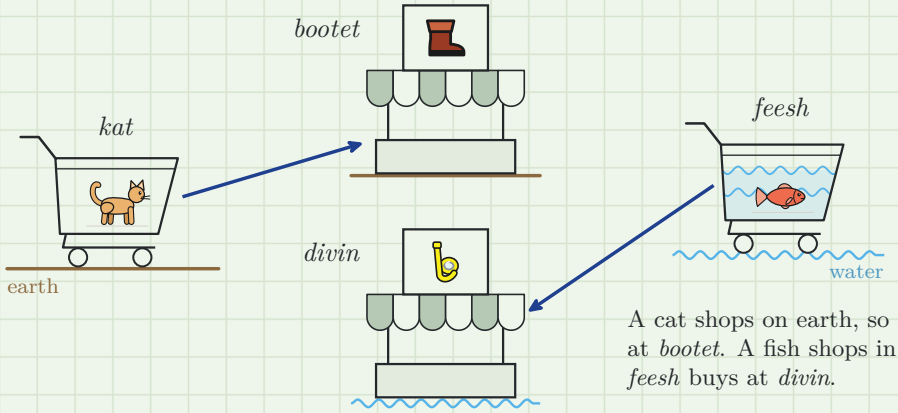






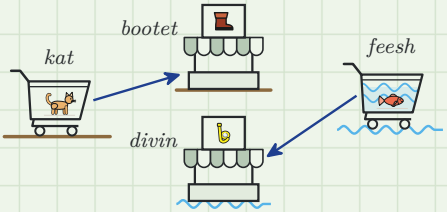
The party matches three keys. Their values go home.

Our sentence makes a small mall: two shoppers, *kat* and *feesh*, and two stands, *bootet* and *divin*.



A cat shops on earth, so *kat* buys at *bootet*. A fish shops in water, so *feesh* buys at *divin*.

Write the arrows into a table S, one row and one column per word in the order of the sentence. A 1 in row i and column j means: word i adds the row of word j.



	bootet	*kat*	*divin*	*feesh*
bootet	**1**	0	0	0
kat	**1**	**1**	0	0
divin	0	0	**1**	0
feesh	0	0	**1**	**1**

S

kat buys at *bootet*: a 1 in row *kat*, column *bootet*.

feesh buys at *divin*: a 1 in row *feesh*, column *divin*.

Every word keeps its own row: a 1 on the diagonal.

Everywhere else 0: no arrow, nothing to add.

Each row of S marks the rows to add. The product SV adds them up: its row i is the sum of the rows of V that row i of S marks with a 1.

kat is now *pussinbootz*, and *feesh* is *divefeesh*: the picture you drew in the game.

$V = E$
for now: part 4 comes back to V

	LEGS	FINS	WINGS	RUNNING	BOOTS	SWIMMING	DIVING	
	0	0	0	0	**1**	0	0	*bootet*
	1	0	0	0	0	0	0	*kat*
	0	0	0	0	0	0	**1**	*divin*
	0	**1**	0	0	0	0	0	*feesh*

S

	bootet	*kat*	*divin*	*feesh*
bootet	**1**	0	0	0
kat	**1**	**1**	0	0
divin	0	0	**1**	0
feesh	0	0	**1**	**1**

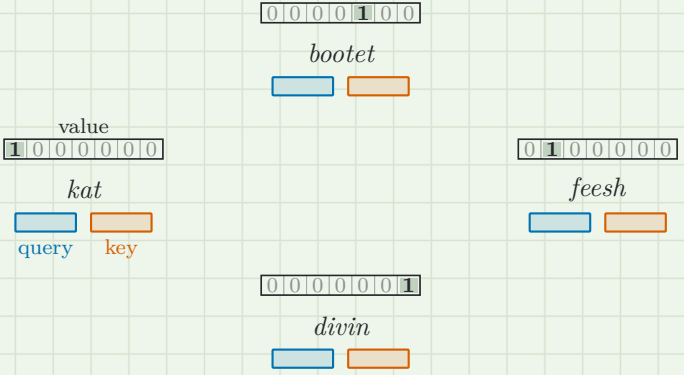
SV

	0	0	0	0	**1**	0	0	*bootet*
	1	0	0	0	**1**	0	0	***pussinbootz***
	0	0	0	0	0	0	**1**	*divin*
	0	**1**	0	0	0	0	**1**	***divefeesh***





Now drop the mall and keep the words where it put them. Every word gets three rows: its value, its query and its key.



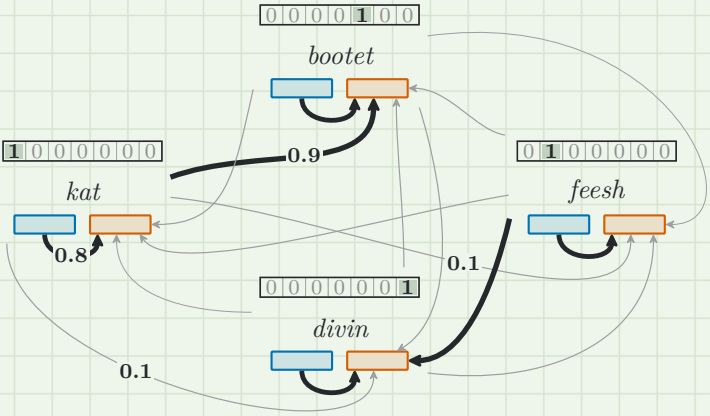
Its **value** is its row of E: what it hands over, as a stand hands over what it sells.

Its **query** says what it looks for, as a shopper does.

Its **key** says what it offers, as the sign of a stand does.

So every word can shop and sell. Part 4 computes queries and keys from E.

The query of a word checks the key of every word, its own included: their inner product is the strength of the arrow. These numbers are made up.



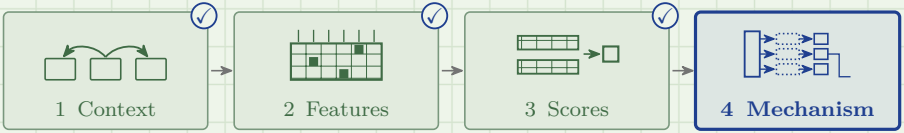
	bootet	*kat*	*divin*	*feesh*
bootet	**0.8**	0.1	0.1	0.0
kat	**0.9**	**0.8**	0.1	0.1
divin	0.1	0.0	**0.8**	0.1
feesh	0.1	0.1	**0.9**	**0.8**

the attention matrix S

Row i holds the arrows of word i. Its large entries sit where the mall had a 1.

Entry 4

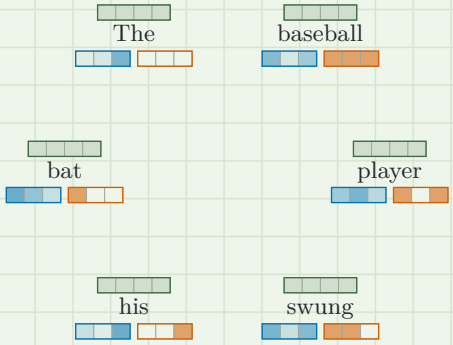
The attention mechanism



So far. On our sentence, every word found the rows to add: the arrows of the mall make the matrix S, the new rows are SV, and the inner products of queries and keys give the strength of every arrow.

Now. The same recipe for any sentence, in five steps. Then the attention mechanism as blocks, with every size a letter.

Take any sentence, here six words, each with its row: its embedding. Every word turns its row into two short rows, a query and a key. All the numbers here are made up.



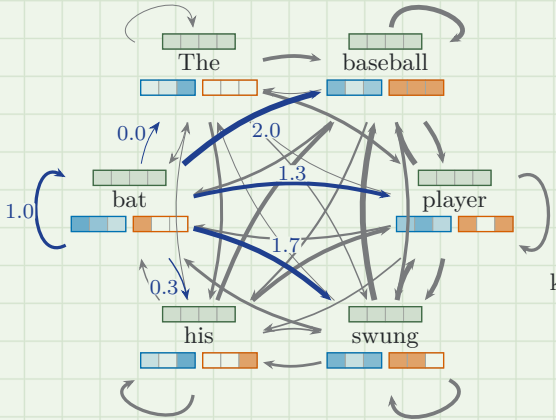
1. **Produce queries and keys**
2. Create the attention graph
3. Softmax the scores into weights
4. Create value vectors
5. Add up the weighted values

The query and the key of **bat**:

query	1.0	0.7	0.3	key	1	0	0

In every row, the darker a cell, the larger its number.

The query of a word meets the key of every word, its own included. Their inner product, the *score*, sets the width of an arrow. All the arrows make the *attention graph*.



1. Produce queries and keys
2. **Create the attention graph**
3. Softmax the scores into weights
4. Create value vectors
5. Add up the weighted values

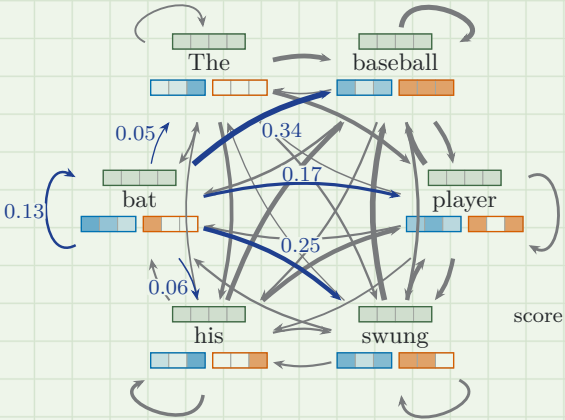
The arrow from **bat** to **baseball**:

query of bat	1.0	0.7	0.3
key of baseball	1	1	1

inner product $1.0 + 0.7 + 0.3 = 2.0$

bat reads **baseball** and **swung** most: here a bat is a club, as at the start of the lecture.

Scores can be any numbers. The softmax raises e to every score of a word and divides by their total: the weights of the word, all positive and adding up to 1.



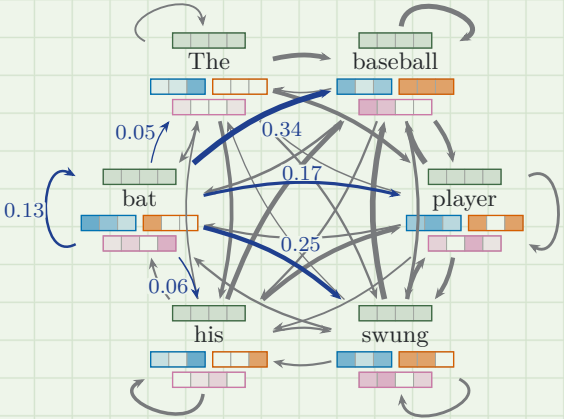
1. Produce queries and keys
2. Create the attention graph
3. **Softmax the scores into weights**
4. Create value vectors
5. Add up the weighted values

$$\text{weight} = \frac{e^{\text{score}}}{\text{sum of } e^{\text{score}} \text{ over all words}}$$

	The	baseball	player	swung	his	bat
score of **bat**	0.0	2.0	1.3	1.7	0.3	1.0
e^{score}	1.00	7.39	3.67	5.47	1.35	2.72
weight	0.05	0.34	0.17	0.25	0.06	0.13

Each e^{score} divided by their total, 21.60.

Every word turns its row into a third short row, its *value*: what the word hands over to every word that reads it.



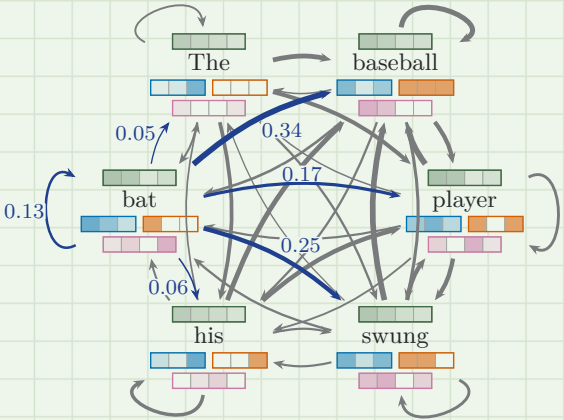
1. Produce queries and keys
2. Create the attention graph
3. Softmax the scores into weights
4. **Create value vectors**
5. Add up the weighted values

The value of **bat**:

value	0.25	0.5	0.0	1.0

A value here has four numbers, as many as an embedding.

The new row of a word, its new embedding, adds up the values of all the words, each multiplied by the weight of its arrow.

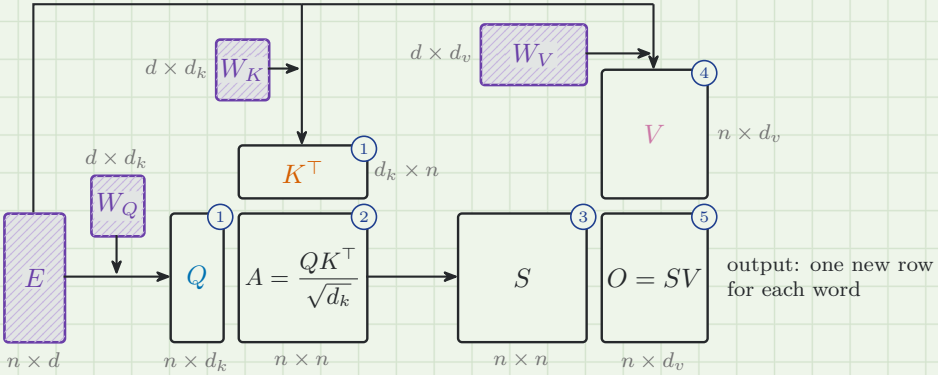


1. Produce queries and keys
2. Create the attention graph
3. Softmax the scores into weights
4. Create value vectors
5. **Add up the weighted values**

0.05×	0.25	0.0	0.0	0.25	The
0.34×	1.0	0.75	0.0	0.0	baseball
0.17×	0.5	0.0	1.0	0.25	player
0.25×	0.75	1.0	0.0	0.5	swung
0.06×	0.0	0.25	0.5	0.0	his
0.13×	0.25	0.5	0.0	1.0	bat
=	0.7	0.6	0.2	0.3	new row of **bat**

Every word does the same with its own weights.

Stack the rows of n words into a block E: the recipe becomes products of matrices and a softmax. Training learns the violet, hatched blocks: E and the three blocks of weights.



We divide by $\sqrt{d_k}$: it keeps the scores in a range where the softmax works.

row i of S: the softmax of row i of A